Guidelines for Research

- Each group should have at least 3 students and no more than 4. Use the following link to provide the group details:

  CECS 543 - Research Group Details - Section 01.xlsx
- Choose your team leader.
- Each group should pick a different topic for research
- Use Overleaf for report writing. An Overleaf link will be provided to each group.
- Select the latest research articles related to Large Language Models and software engineering published in 2023 or later.

Topics for Research

1. Software Engineering for AI
2. Generative AI and Software Engineering
3. Use of GenAI for requirement analysis
4. Use of GenAI for software maintenance
5. Use of GenAI for software security
6. Use of GenAI for quality assurance
7. Use of GenAI for healthcare systems
8. Security of Generative AI
9. Enabling software engineering for process improvement
10. Quantum computing for software development

S. No.	Research Problems (The details can be found starting on page 5)
01	An Agentic Early-Warning System for High-Risk Commits: Fusing Code Complexity and Issue-Tracker Patterns for Cross-Domain Defect Prediction
02	Cross-Project Defect Prediction for Unlabeled Target Repositories via Transfer Learning
03	Classifying Requirements for Automated Traceability: Categorization and Trace-Link Recovery
04	Predicting Task Blockers and Delay Cascades in Agile Sprints via Dependency-Network Analysis
05	Beyond Single-Comment Toxicity Detection: Sentiment-Trajectory Modeling for Developer Fatigue and Contributor Retention
06	Defect-Aware, Context-Sensitive Task Allocation for Bug Triage in Agile Software Development
07	Toward Real-Time Carbon-Smell Detection in Python: Micro-Level Software Energy Budgeting
08	Eco-Certified AI Model Selection in Software Design: Environmental Footprint Budgeting for Third-Party LLM Integration

Tentative Schedule for Experimental Work

Due dates	Event	Points
Sep. 4th	Group formation and team lead selection	0.5
Sep. 11th	Picking a research article to replicate	0.5
Oct. 9th	Replicating the results	1
Nov. 20th	-Try to improve the results and design a framework (generic steps to do something). -Report submission: the report will be based on the following: 1. Title • Title: Concise, specific, and informative. • Author Information: Names, affiliations, and contact information. 2. Abstract • A brief summary (150-250 words) of the research, covering: o Context of the research. o Problem statement. o methodology. o Key findings. o Implication of research 3. Keywords • 4-6 keywords relevant to your research. 4. Introduction • Background: Context and importance of the research topic. • Research Problem/Objective: • Research Aim and Objectives: A clear statement of what the study aims to achieve. • Significance of the Study: Why is the study important? What contribution does it make? • Research Questions/Hypotheses: Specific questions or hypotheses to be explored. • Scope of the Study: Boundaries of the research (who, where, when). 5. Proposed Methodology 6. Case study/ Data Preparation 7. Results and Discussion 8. Limitations 9. Implication of Research in Other Areas 10. Future Work 11. Conclusions 12. References	3

Here are some tips for writing a report:

-Abstract should not have any citations

-The introduction section should have 3-4 citations

-All acronyms should be defined on first appearance

-Each table and figure should be described

-The starting line of the first paragraph of each section should be an introduction of that section, and the last line of the final paragraph should be related to the next section.

-Highlight the novelty and significance of your work

-Highlight the achievements

Problem 1

An Agentic Early-Warning System for High-Risk Commits: Fusing Code Complexity and Issue-Tracker Patterns for Cross-Domain Defect Prediction

I. INTRODUCTION

Modern continuous-integration pipelines generate a continuous stream of commits and pull requests, but quality-assurance resources are never sufficient to manually inspect every change with equal rigor. When a change combines high code complexity — deep nesting, high coupling, large churn — with certain issue-tracker patterns, such as repeated reopenings or a history of defect-prone files being touched together, the likelihood of a severe downstream defect rises sharply. Developers, however, typically receive no automated warning of this compounded risk at the moment they push a change or open a pull request.

This project designs and evaluates an agentic early-warning component that sits inside a CI/PR workflow and flags high-risk commits before they are merged, by fusing code-complexity metrics with issue-tracker and pull-request metadata. It builds directly on the established Just-in-Time Software Defect Prediction (JIT-SDP) literature, but extends it to a cross-domain, pull-request-linked setting using the GHPR dataset, which pairs object-oriented code metrics (CBO, WMC, DIT, RFC, LCOM and related complexity measures) with defect labels mined from real GitHub pull requests.

II. LITERATURE REVIEW

Just-in-time defect prediction reframes classical, file- or module-level defect prediction as a change-level problem, reasoning about risk at the moment a commit is made rather than over a static code snapshot (Kim, Whitehead, & Zhang, 2008). Kamei et al. (2013) formalized this shift with fourteen change-level metrics spanning five dimensions — diffusion, size, purpose, history and developer experience — combined through an effort-aware logistic-regression model; this feature set remains the de-facto baseline against which newer approaches are compared.

Subsequent work has focused on raising predictive power and granularity. JITLine (Pornprasit & Tantithamthavorn, 2021) combined interpretable features with fine-grained, line-level risk localization, reporting large gains in F-measure and cost-effectiveness over deep-learning baselines such as CC2Vec and DeepJIT while running substantially faster. In parallel, a growing body of work has scrutinized the quality of JIT-SDP ground truth itself: because bug-inducing commits are usually identified indirectly through SZZ-style linkage from later bug-fixing commits, labels are frequently noisy, an issue highlighted by the ReDef dataset and calibration studies (2025), and systematically catalogued in Damevski et al.'s survey of the field.

For this project's specific dataset, Xu, Wang and Ai (2021) constructed the GHPR dataset explicitly to support semantic and graph-based defect prediction, pairing object-oriented complexity metrics with pull-request-level defect outcomes across thousands of learning instances; later CodeBERT-based JIT-SDP work has reused GHPR-style datasets in cross-project evaluations, and empirical studies of review-related features (reviewer count, discussion length, response latency) show that such process metadata meaningfully complements code-complexity metrics rather than substituting for them — directly supporting the fused approach proposed here.

III. RESEARCH QUESTIONS

RQ1. *To what extent does combining code-complexity metrics (WMC, RFC, LCOM, nesting depth) with PR/issue-tracker patterns (reopen count, review latency, discussion length) improve high-risk-commit detection over either feature family alone, on GHPR?*

RQ2. *Can a lightweight early-warning agent deployed at pull-request time reduce reviewer triage effort at a fixed inspection budget, relative to unranked review?*

RQ3. *How well does a model trained on one GHPR project generalize, without retraining, to a different project in a cross-domain evaluation?*

REFERENCES

[1] Kamei, Y., Shihab, E., Adams, B., Hassan, A. E., Mockus, A., Sinha, A., & Ubayashi, N. (2013). A large-scale empirical study of just-in-time quality assurance. IEEE Transactions on Software Engineering, 39(6), 757–773.

[2] Kim, S., Whitehead, E. J., & Zhang, Y. (2008). Classifying software changes: Clean or buggy? IEEE Transactions on Software Engineering, 34(2), 181–196.

[3] Pornprasit, C., & Tantithamthavorn, C. (2021). JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction. arXiv:2103.07068.

[4] Damevski, K., et al. (2022). A systematic survey of just-in-time software defect prediction. ACM Computing Surveys.

[5] Xu, J., Wang, F., & Ai, J. (2021). Defect prediction with semantics and context features of codes based on graph representation learning. IEEE Transactions on Reliability. [GHPR dataset]

[6] Probing pre-trained language models on code changes: Insights from ReDef, a high-confidence just-in-time defect prediction dataset (2025). arXiv:2509.09192.

[7] Feature sets in just-in-time defect prediction: An empirical evaluation (2022). arXiv:2209.13978.

Problem 2

Cross-Project Defect Prediction for Unlabeled Target Repositories via Transfer Learning

I. INTRODUCTION

New or historically unmonitored software projects typically lack the labeled defect history that within-project defect-prediction models depend on, leaving teams unable to prioritize testing effort until enough bugs have already surfaced in production. Cross-Project Defect Prediction (CPDP) addresses this by training a model on labeled data from one or more source projects and applying it to an unlabeled target project, but the approach is fundamentally limited by distribution shift: source and target projects rarely share the same feature distribution, code style, or defect base-rate.

This project designs and evaluates a CPDP pipeline for locating defects in unlabeled target repositories drawn from the MTP dataset, framing the task explicitly as a transfer-learning problem and comparing several families of distribution-adaptation techniques.

II. LITERATURE REVIEW

Nam, Pan and Kim (2013) were among the first to cast CPDP as a transfer-learning problem, proposing TCA+ to align the marginal feature distributions of source and target projects before training a classifier, and demonstrating that appropriately transformed cross-project data can approach within-project performance. Because CPDP targets are, by definition, unlabeled, Nam and Kim (2015) proposed CLAMI, which estimates pseudo-labels for the target via clustering and magnitude-based thresholding rather than assuming any manual annotation is available — a design directly applicable to genuinely unlabeled data such as MTP's target repositories.

A recurring critique in the CPDP literature is that many transfer techniques correct only marginal distribution differences while ignoring conditional distribution mismatch, which several studies show degrades classification quality; balanced distribution adaptation approaches were introduced specifically to correct both simultaneously and have reported consistent gains over marginal-only baselines. Because software defect datasets are also typically class-imbalanced, Tong, Liu, Wang and Li (2019) combined a transfer-learning-oriented oversampling technique with feature-weighted transfer Naive Bayes, reporting large improvements in G-measure and Matthews correlation coefficient across eleven public defect datasets when imbalance and feature-importance were addressed jointly.

Sotto-Mayor and Kalech's 2024 survey organizes 44 CPDP methods into a transfer-learning design-space taxonomy (instance-based, feature-based, parameter-based and relational approaches), providing a structured basis for selecting candidate techniques. At the same time, retrospective and data-selection studies caution that CPDP gains are not automatic: careful source-project or instance selection, rather than simple domain similarity, has repeatedly been shown to be the dominant driver of cross-project performance, and some benchmark comparisons find CPDP does not uniformly outperform simple baselines — a reminder that rigorous baseline comparison is essential to this project's evaluation.

III. RESEARCH QUESTIONS

RQ1. *Which transfer strategy — instance-weighting, marginal/conditional distribution alignment, or CLAMI-style pseudo-labeling — most accurately predicts defect locations in an unlabeled MTP target project?*

RQ2. *Does jointly correcting marginal and conditional distribution differences outperform marginal-only adaptation, and by how much?*

RQ3. *How does combining class-imbalance-aware resampling with transfer learning affect prediction quality (G-mean, AUC) relative to the same transfer technique without resampling?*

REFERENCES

[1] Nam, J., Pan, S. J., & Kim, S. (2013). Transfer defect learning. Proceedings of ICSE 2013.

[2] Nam, J., & Kim, S. (2015). CLAMI: Defect prediction on unlabeled datasets. Proceedings of ASE 2015, 452–463.

[3] Sotto-Mayor, B., & Kalech, M. (2024). A survey on transfer learning for cross-project defect prediction.

[4] Tong, H., Liu, B., Wang, S., & Li, Q. (2019). Transfer-learning oriented class imbalance learning for cross-project defect prediction. arXiv:1901.08429.

[5] Cross project defect prediction via balanced distribution adaptation based transfer learning (2019). Journal of Computer Science and Technology.

[6] Training data selection for imbalanced cross-project defect prediction (2021). Computers & Electrical Engineering.

[7] An empirical study on the effectiveness of data resampling approaches for cross-project software defect prediction (2022). IET Software.

Problem 3

Classifying Requirements for Automated Traceability: Categorization and Trace-Link Recovery

I. INTRODUCTION

Requirements are typically written in informal natural language, which makes it difficult to automatically and reliably link them to the system components that implement them. Left unmanaged, this gap between stated intent and implemented architecture contributes to feature scope creep and to traceability matrices that are expensive to build and quickly go stale as a project evolves.

This project builds an automated pipeline that first categorizes requirements (functional vs. non-functional, with non-functional sub-classes) and then recovers trace links between requirements and code, using the PROMISE requirements-classification dataset for categorization and fine-grained traceability techniques such as Hey et al.'s FTLR as a methodological baseline for link recovery.

II. LITERATURE REVIEW

The PROMISE NFR dataset — 625 requirements labeled as functional or across eleven non-functional sub-classes — has served for over a decade as the field's most widely reused, reproducible benchmark for requirements classification, and continues to underpin recent machine-learning and transformer-based classification studies. A complementary resource, the PURE dataset (Ferrari, Spagnolo, & Gnesi, 2017), supplies 79 real, publicly available requirements documents and over 34,000 sentences, enabling classification and ambiguity-detection research on genuine specification text.

On the traceability side, Guo, Cheng and Cleland-Huang (2017) were early to show that deep learning — recurrent neural networks capturing semantic relationships — could outperform classical information-retrieval baselines for linking requirements to design artifacts. This line advanced further with pre-trained transformer models: Lin et al.'s (2021) 'Traceability Transformed' fine-tuned BERT for trace-link generation with markedly higher accuracy than earlier IR and RNN approaches, and a 2022 follow-up extended this via transfer learning from open-world data to reduce the need for large project-specific labeled trace links.

Because most requirements-to-code approaches operate at a coarse granularity, Hey, Chen, Weigelt and Tichy (2021) proposed FTLR, operating instead at the level of individual requirement sentences and source-code methods, reporting strong recovery accuracy without the very large manually labeled training sets that fully supervised models such as TRAIL require. A 2026 study further demonstrates that the underlying quality of the requirements text itself — its ambiguity and completeness — materially affects trace-link recovery performance, motivating a combined categorization-plus-quality-aware approach for this project.

III. RESEARCH QUESTIONS

RQ1. *How accurately can a fine-tuned transformer classify natural-language requirements into functional/non-functional categories on PROMISE, compared to classical ML baselines?*

RQ2. *Does combining fine-grained (FTLR-style) trace-link recovery with automated categorization improve traceability-matrix precision and recall over coarser class-level baselines?*

RQ3. *To what extent does requirement-writing quality (ambiguity, incompleteness) on PURE/PROMISE mediate the accuracy of downstream trace-link recovery?*

REFERENCES

[1] Ferrari, A., Spagnolo, G. O., & Gnesi, S. (2017). PURE: A dataset of public requirements documents. Proceedings of RE 2017, 502–505.

[2] Guo, J., Cheng, J., & Cleland-Huang, J. (2017). Semantically enhanced software traceability using deep learning techniques. Proceedings of ICSE 2017, 3–14.

[3] Lin, J., Liu, Y., Zeng, Q., Jiang, M., & Cleland-Huang, J. (2021). Traceability transformed: Generating more accurate links with pre-trained BERT models. Proceedings of ICSE 2021, 324–335.

[4] Lin, J., Poudel, A., Yu, W., Zeng, Q., Jiang, M., & Cleland-Huang, J. (2022). Enhancing automated software traceability by transfer learning from open-world data. arXiv:2207.01084.

[5] Hey, T., Chen, F., Weigelt, S., & Tichy, W. F. (2021). Improving traceability link recovery using fine-grained requirements-to-code relations. Proceedings of ICSME 2021, 12–22.

[6] Synergistic enhancement of requirement-to-code traceability: A framework combining LLM-based data augmentation and an advanced encoder (2025). arXiv:2509.20149.

[7] How requirements quality makes (or breaks) traceability link recovery (2026). arXiv:2606.11834.

[8] PROMISE_exp / PROMISE NFR repository. http://promisedata.org/repository

Problem 4

Predicting Task Blockers and Delay Cascades in Agile Sprints via Dependency-Network Analysis

I. INTRODUCTION

In agile sprints, a delay on one user story or bug fix rarely stays contained: because tasks are frequently linked through dependency relations such as blocked-by, same-developer, or shared-component edges, an unnoticed delay can cascade through the dependency network and derail the release schedule as a whole. Despite this, most sprint-tracking tools surface only aggregate metrics like burndown and velocity, with little support for identifying which specific tasks are structurally positioned to trigger downstream cascades.

This project builds a graph-based bottleneck predictor over a project's task-dependency network, using an agile sprint-planning dataset and building directly on Choetkiertikul et al.'s foundational work on predicting delay in iterative software development.

II. LITERATURE REVIEW

Choetkiertikul, Dam, Tran and Ghose framed software delay prediction as a classification problem over a network of dependent issues rather than an independent per-issue problem, releasing a benchmark of 8,206 JBoss issues connected through twelve relation types and showing that a task's position within this dependency network carries predictive signal beyond its own features. This benchmark was later reused by Pham, Tran and Venkatesh's Column Networks, a relational deep-learning architecture for collective classification that propagates information across an issue's dependency neighbors and improved delay-prediction accuracy over feature-only baselines.

More recent, application-oriented literature reflects growing industry interest: knowledge-graph-based frameworks for sprint-performance prediction (2026) hypothesize that task-dependency depth and betweenness centrality are structurally associated with sprint completion rate, directly motivating a graph-centrality feature set. Separately, a Bayesian delay-pattern model evaluated on real enterprise epics at a large financial-services company showed that dynamically re-estimating delay risk using phase-aware delay patterns outperformed static effort-estimation baselines, suggesting bottleneck warnings should be updated continuously through a sprint.

Industry-facing practitioner literature on AI-assisted agile delivery (2025–2026) further documents growing use of anomaly-detection on velocity and cycle-time signals to flag emerging bottlenecks, though largely without the rigorous empirical validation found in the academic delay-prediction literature — a gap this project's evaluation is positioned to help close.

III. RESEARCH QUESTIONS

RQ1. *Do graph-structural features (dependency depth, betweenness centrality, blocked-by fan-out) predict downstream task delay more accurately than issue-only text/metadata features?*

RQ2. *Can a collective/relational classifier (e.g., a GNN over the dependency network) outperform an independent per-issue classifier at identifying delay-cascading bottlenecks?*

RQ3. *How early in a sprint can the model issue actionable bottleneck warnings while maintaining acceptable precision?*

REFERENCES

[1] Choetkiertikul, M., Dam, H. K., Tran, T., & Ghose, A. (2017). Predicting delivery capability in iterative software development. IEEE Transactions on Software Engineering.

[2] Pham, T., Tran, T., & Venkatesh, S. (2017). Column networks for collective classification. arXiv:1609.04508.

[3] A knowledge graph–based framework for predicting sprint performance in agile software projects (2026). European Journal of Business and Management Research.

[4] Dynamic prediction of delays in software projects using delay patterns and Bayesian modeling (2023). arXiv:2309.12449.

[5] Top 5 AI techniques for predicting project delays (2025). Agileseekers (practitioner report).

[6] The role of AI in identifying and removing agile bottlenecks (2026). Agileseekers (practitioner report).

Problem 5

Beyond Single-Comment Toxicity Detection: Sentiment-Trajectory Modeling for Developer Fatigue and Contributor Retention

I. INTRODUCTION

Open-source and enterprise software projects alike suffer meaningful contributor attrition linked to toxic communication and sustained work overload in issue threads and code-review discussions. Existing toxicity detectors are effective at flagging individual toxic comments but rarely model how a contributor's sentiment evolves over time, which limits their usefulness for anticipating burnout or disengagement before it happens.

This project extends the CMUSTRUDEL toxicity-detector line of research into a longitudinal sentiment-trajectory model that aims to predict contributor fatigue and attrition risk, rather than only classifying isolated comments.

II. LITERATURE REVIEW

Raman, Cao, Tsvetkov, Kästner and Vasilescu (2020) built the first classifier specifically tuned to detect toxicity in open-source GitHub issue discussions, combining pre-trained detectors of negative sentiment, impoliteness and abusive language (including Google's Perspective API) and adapting them to the linguistic norms of software-engineering discourse. Their study also found that corporate-sponsored projects tend to be less toxic than purely volunteer-run ones, and that contributors with a history of toxic comments are markedly more likely to repeat that behavior — an early hint that toxicity has a temporal, trajectory-like structure rather than being purely episodic.

Follow-up empirical work found that contributors' behavioral responses to toxicity — including attrition — are shaped by repeated rather than one-off negative interactions, and that contributors with consistently positive sentiment show the highest productivity and interaction levels. Toxicity detection has since been extended beyond issue threads to code-review comments, including explainable, span-level detection approaches such as ToxiSpanSE, which localize exactly which phrases in a review comment are toxic rather than only classifying the comment as a whole.

A 2026 systematic literature review of ML models for early burnout detection in software engineering synthesizes this literature into four recurring target classes — sentiment/emotion, attrition, toxic relationship, and combined emotion/stress detection — and finds that text-based inputs remain the dominant and most practical data modality. Earlier work linking commit-message sentiment to downstream code-quality outcomes (Guzman, Azócar, & Li, 2014) further suggests a sentiment trajectory may correlate with code-quality trends, offering a secondary outcome variable worth exploring here.

III. RESEARCH QUESTIONS

RQ1. *Can a sentiment/toxicity trajectory model — rather than a single-comment classifier — predict contributor disengagement within a defined follow-up window?*

RQ2. *Which trajectory features (slope, volatility, frequency of toxic episodes, response latency) are most predictive of attrition on the CMUSTRUDEL-derived corpus?*

RQ3. *Does the trajectory-based model reduce false positives relative to a single-comment toxicity classifier baseline?*

REFERENCES

[1] Raman, N., Cao, M., Tsvetkov, Y., Kästner, C., & Vasilescu, B. (2020). Stress and burnout in open source: Toward finding, understanding, and mitigating unhealthy interactions. Proceedings of ICSE-NIER 2020.

[2] “Did you miss my comment or what?”: Understanding toxicity in open source discussions (2022).

[3] ToxiSpanSE: An explainable toxicity detection in code review comments (2023). arXiv:2307.03386.

[4] Analyzing toxicity in open source software communications (2024). arXiv:2412.13133.

[5] Machine learning models for the early detection of burnout in software engineering: A systematic literature review (2026). arXiv:2603.23063.

[6] Guzman, E., Azócar, D., & Li, Y. (2014). Sentiment analysis of commit comments in GitHub: An empirical study. Proceedings of MSR 2014, 352–355.

Problem 6

Defect-Aware, Context-Sensitive Task Allocation for Bug Triage in Agile Software Development

I. INTRODUCTION

Assigning bugs primarily on the basis of developer workload, without regard to a developer's familiarity with the affected code module, frequently produces inefficient fixes and costly reassignment chains, often referred to as bug tossing. This project builds a context-aware, defect-aware task recommender that allocates bugs by combining developer expertise signals with knowledge of which modules are historically error-prone, evaluated on the TAWOS agile issue-tracking dataset.

II. LITERATURE REVIEW

Automated bug triage as a supervised-learning recommendation problem was established by Anvik, Hiew and Murphy (2006), who framed 'who should fix this bug' as a text-classification task over historical bug reports and their eventual resolvers. Matter, Kuhn and Nierstrasz (2009) refined the notion of developer expertise by modeling it from the vocabulary found in a developer's own source-code contributions rather than only from bug-report text, testing the approach on over 130,000 Eclipse bug reports.

A parallel line of work models bug reassignment explicitly through tossing graphs: Bhattacharya and Neamtiu's fine-grained, incremental tossing-graph approach, and later machine-learning-augmented tossing-graph models, aimed to shorten the chain of reassignments a bug undergoes before reaching the developer who ultimately fixes it. Complementary recommender designs have combined topic modeling of bug 'concepts' with social-network measures of developer centrality to rank candidate assignees, and a later expertise-score ranking approach integrated priority, versatility and average fix-time into a single composite developer score.

More recent work has begun to combine developer-history-based relevance scoring with explainable AI techniques, addressing long-standing critiques that earlier ML triage systems functioned as opaque black boxes, and a large systematic review of triage research documents a persistent gap between the strong accuracy figures reported in academic studies and comparatively modest gains observed in registered industrial case studies — a caution this project's evaluation should take seriously.

III. RESEARCH QUESTIONS

RQ1. *Does an expertise model derived from a developer's historical commit/module-touch vocabulary, rather than workload alone, reduce bug reassignment (tossing) length on TAWOS?*

RQ2. *How does a topic/concept-based recommender compare to a tossing-graph-based recommender in resolution time and first-assignment accuracy?*

RQ3. *Does incorporating module-level defect-proneness into the recommender improve fix rate and reduce time-to-resolution over an expertise-only baseline?*

REFERENCES

[1] Anvik, J., Hiew, L., & Murphy, G. C. (2006). Who should fix this bug? Proceedings of ICSE 2006, 361–370.

[2] Matter, D., Kuhn, A., & Nierstrasz, O. (2009). Assigning bug reports using a vocabulary-based expertise model of developers. Proceedings of MSR 2009, 131–140.

[3] Bhattacharya, P., & Neamtiu, I. (2010). Fine-grained incremental learning and multi-feature tossing graphs to improve bug triaging. Proceedings of ICSM 2010, 1–10.

[4] Bhattacharya, P., Neamtiu, I., & Shelton, C. R. (2012). Automated, highly-accurate, bug assignment using machine learning and tossing graphs. Journal of Systems and Software, 85(10), 2275–2292.

[5] Xi, S.-Q., Yao, Y., Xiao, X.-S., Xu, F., & Lv, J. (2019). Bug triaging based on tossing sequence modeling. Journal of Computer Science and Technology, 34(5), 942–956.

[6] Ranking of software developers based on expertise score for bug triaging (2019). Information and Software Technology.

[7] Improving bug assignment and developer allocation in software engineering through interpretable machine learning models (2023). Big Data and Cognitive Computing, 7(3), 128.

[8] Triage in software engineering: A systematic review of research and practice (2025). arXiv:2511.08607.

Problem 7

Toward Real-Time Carbon-Smell Detection in Python: Micro-Level Software Energy Budgeting

I. INTRODUCTION

Developers currently have very little real-time, in-IDE feedback about how individual functions, loops, or library calls affect the energy consumption and CO2 output of the code they are writing; most existing tooling reports only aggregate, project- or run-level totals well after the code has been executed. This project builds a real-time (or near-real-time) 'energy smell' detector for Python that surfaces actionable, function-level feedback as code is written, instrumented using the eco2AI emissions tracker.

II. LITERATURE REVIEW

The empirical study of software energy consumption traces back to Hindle's foundational 'green mining' methodology, which linked software change history directly to measured power consumption, and to the GreenMiner framework, a hardware-based test-bed that automated energy measurement of running applications. Early energy-smell catalogs emerged from the mobile/Android space, where studies of design-pattern energy usage and of energy 'bad smells' with associated refactoring strategies established the idea that specific, recognizable code patterns reliably correlate with excess energy draw.

This static-analysis-rule-catalogue approach has since been formalized into practical tooling: ecoCode, described in a 2022 ERCIM News overview, is a SonarQube plugin that statically detects dozens of energy code smells, initially for Android/Java projects but explicitly extending toward Python. A large 2026 study went further by empirically deriving an energy-smell taxonomy directly from measurement: profiling more than 21,000 functionally equivalent Python code pairs for energy, time and memory, and finding that the large majority of samples exhibit multiple co-occurring smells and that memory-related smells yield the greatest per-fix energy savings — a labeled taxonomy directly reusable as ground truth for this project's detector.

Complementary work at the commit level, EnergyTrackr, demonstrated that energy regressions and their associated code-change patterns can be detected automatically across a project's commit history, evaluated on 3,232 commits from three Java projects — a methodological template transferable to a Python, IDE-integrated setting. Underlying all of these approaches is measurement infrastructure such as eco2AI and CodeCarbon, which this project will wrap with a live linting/feedback layer.

III. RESEARCH QUESTIONS

RQ1. *Which Python-level code patterns (loop structures, library calls, memory-allocation patterns) correlate most strongly with measurable increases in energy/CO2 output, as measured via real-time eco2AI instrumentation?*

RQ2. *Can a lightweight static/dynamic hybrid detector flag energy-smelly code in near real time (within an IDE linting loop) with acceptable precision against the empirically derived energy-smell taxonomy?*

RQ3. *How much energy/CO2 reduction is achievable by applying the detector's suggested refactorings to real Python functions, relative to unmodified baselines?*

REFERENCES

[1] Hindle, A. (2012). Green mining: A methodology of relating software change to power consumption. Proceedings of MSR 2012, 78–87.

[2] Hindle, A., Wilson, A., Rasmussen, K., Barlow, E. J., Campbell, J. C., & Romansky, S. (2014). GreenMiner: A hardware based mining software repositories software energy consumption framework. Proceedings of MSR 2014, 12–21.

[3] Sahin, C., Cayci, F., Gutiérrez, I. L. M., Clause, J., Kiamilev, F., Pollock, L., & Winbladh, K. (2012). Initial explorations on design pattern energy usage. Proceedings of GREENS 2012, 55–61.

[4] Software energy efficiency: New tools for developers — ecoCode (2022). ERCIM News.

[5] Watts this smell: A comprehensive taxonomy of software energy smells (2026). Zenodo.

[6] Systematic detection of energy regression and corresponding code patterns in Java projects — EnergyTrackr (2026). arXiv:2604.19373.

[7] Budennyy, S., et al. (2022). eco2AI: Carbon emissions tracking of machine learning models. GitHub: sb-ai-lab/eco2AI.

Problem 8

Eco-Certified AI Model Selection in Software Design: Environmental Footprint Budgeting for Third-Party LLM Integration

I. INTRODUCTION

Software architects who integrate third-party large language models into AI-powered features currently choose model checkpoints primarily on the basis of accuracy, cost and latency, with no standardized, verifiable metric to weigh ecological impact alongside those criteria — and no direct way to measure the footprint of proprietary, API-only models in the first place. This project builds a model-selection framework that budgets environmental footprint for third-party LLM integration decisions, grounded in the EcoLogits estimation approach and the Green Software Foundation's Software Carbon Intensity (SCI) methodology.

II. LITERATURE REVIEW

The field's foundational framing comes from Schwartz, Dodge, Smith and Etzioni's (2020) distinction between 'Red AI,' which pursues accuracy at any computational cost, and 'Green AI,' which treats efficiency as a first-class research objective alongside accuracy. Luccioni, Viguier and Ligozat (2022) operationalized this for a specific, large open model by producing one of the first rigorous end-to-end carbon-footprint estimates for BLOOM, a 176-billion-parameter language model, covering training, inference and embodied hardware emissions.

Because proprietary models such as GPT-4 or Claude cannot be directly instrumented for energy draw, EcoLogits was developed to estimate their environmental impact through regression-based energy and latency prediction rather than hardware telemetry; a 2025 benchmarking study situates EcoLogits alongside several other joint accuracy-energy evaluation efforts, providing candidate scoring functions this project's 'environmental budget' score can draw on. Separately, a large-scale mining study of 1,417 Hugging Face models found no significant correlation between model performance and carbon emissions, suggesting low-carbon, high-accuracy options already exist and can be surfaced by a selection tool.

Complementing this, recent studies of small language models show that fine-tuned SLMs can match larger LLMs' performance on several tasks while cutting inference-time emissions by an order of magnitude or more, supporting an SLM-first recommendation strategy. Finally, the Green Software Foundation's Software Carbon Intensity specification — now standardized as ISO/IEC 21031, with a dedicated 'SCI for AI' extension — provides the standardized, rate-based 'carbon per functional unit' formalism this project's tool will use to make third-party model choices auditable and comparable across vendors.

III. RESEARCH QUESTIONS

RQ1. *How reliably can regression-based estimators (EcoLogits-style) approximate the energy/carbon footprint of proprietary, API-only LLMs compared to available ground-truth or vendor-disclosed figures?*

RQ2. *Can a standardized environmental-budget score, based on the SCI/SCI-for-AI methodology, meaningfully differentiate candidate third-party models without materially sacrificing task accuracy?*

RQ3. *To what extent can smaller or open models substitute for larger proprietary LLMs in a representative software feature while maintaining acceptable accuracy and reducing estimated footprint?*

REFERENCES

[1] Schwartz, R., Dodge, J., Smith, N. A., & Etzioni, O. (2020). Green AI. Communications of the ACM, 63(12), 54–63.

[2] Luccioni, A. S., Viguier, S., & Ligozat, A.-L. (2022). Estimating the carbon footprint of BLOOM, a 176B parameter language model.

[3] Smart but costly? Benchmarking LLMs on functional accuracy and energy efficiency (2025). arXiv:2511.07698.

[4] Green metrics for AI: A hybrid strategy for environmental impact assessment (2025). ScienceDirect.

[5] Innovating for tomorrow: The convergence of SE and Green AI (2024). arXiv:2406.18142.

[6] Emissions and performance trade-off between small and large language models (2026). arXiv:2601.08844.

[7] Green Software Foundation. Software Carbon Intensity (SCI) Specification, ISO/IEC 21031, and the SCI for AI extension. https://sci.greensoftware.foundation/